

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: ARTIFICIAL NEURAL NETWORKS)
(SUBJECT CODE:24SBT113)

AN INDIVIDUAL TASK REPORT ON
“Design and Implementation of a Perceptron Model for
Binary Classification Using Manual Calculation and
Spreadsheet Application”

Submitted in the partial fulfillment of the requirements for the Fourth semester

BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning

Submitted By,
SREERANJINI P
01SU24AI102

UNDER THE GUIDANCE OF
Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2025-26

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

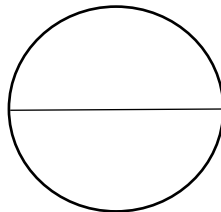


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Sreeranjini P** (**01SU24AI102**), has satisfactorily completed the assessment (Individual Task Report) in **Artificial Neural Networks (24SBT113)** prescribed by the Srinivas University for the 4th semester B. Tech course during the year **2025-26**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B



TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction to Perceptron	2
3	Structure of the Perceptron Model	2
4	Mathematical Model of the Perceptron	4
5	Activation Function in Perceptron	5
6	Perceptron Learning Law	6
7	Complete AND Gate Training	9
8	Spreadsheet Implementation of Perceptron	12
9	Decision Boundary	14
10	Advantages and Limitations	14
11	Conclusion	16
12	References	16

1.Abstract

The Perceptron is one of the earliest supervised learning algorithms developed for solving binary classification problems. Introduced in 1958 by **Frank Rosenblatt**, it is considered the foundation of artificial neural networks. The perceptron model is inspired by the biological neuron and operates by computing a weighted sum of input features, adding a bias term, and applying a step activation function to produce a binary output.

This report focuses on building and understanding a Perceptron model through both manual (paper-based) calculations and spreadsheet implementation. The mathematical representation of the perceptron is explained in detail, including the weighted sum equation and the Perceptron Learning Law. The learning rule enables the model to update its weights iteratively based on classification errors, allowing it to improve performance over time.

To demonstrate practical application, the perceptron is trained to solve a binary classification problem using the AND logic gate dataset. The training process involves initializing weights, calculating outputs, computing errors, and updating weights until convergence is achieved. The concept of a linear decision boundary is also discussed to illustrate how the perceptron separates two classes when the data is linearly separable.

Additionally, the report explains how the same training process can be implemented using spreadsheet formulas, enabling learners to observe weight updates and error reduction step-by-step without requiring programming knowledge. Although the perceptron has limitations—such as its inability to handle non-linearly separable problems—it remains an essential model for understanding the fundamentals of neural networks and machine learning.

2.INTRODUCTION

Artificial Intelligence and Machine Learning have transformed modern technology by enabling machines to learn from data and make intelligent decisions. One of the earliest and most significant developments in this field is the **Perceptron**, a supervised learning algorithm designed for binary classification. Introduced in 1958 by Frank Rosenblatt, the perceptron marked the beginning of artificial neural network research and established the foundation for modern deep learning systems.

In machine learning, classification refers to the task of assigning input data into predefined categories. When there are only two possible categories, the problem is known as binary classification. Examples include spam detection (spam or not spam), medical diagnosis (disease or no disease), loan approval (approved or rejected), and pass/fail prediction. The perceptron is specifically designed to solve such two-class problems when the data is linearly separable.

The perceptron model is inspired by the biological neuron found in the human brain. A biological neuron receives signals from other neurons through dendrites, processes them in the cell body, and transmits an output signal through the axon. Similarly, the artificial perceptron receives multiple input values, multiplies each by a corresponding weight, adds a bias term, and processes the result through an activation function to produce a final output. This simplified mathematical model of a neuron demonstrates how learning can occur through numerical adjustments of weights.

Although the perceptron is simple in structure and limited in capability—particularly its inability to solve non-linearly separable problems such as the XOR problem—it remains extremely important in the history of artificial intelligence. The limitations of the perceptron later motivated the development of multi-layer neural networks and advanced learning algorithms, which now power modern deep learning systems.

3. STRUCTURE OF THE PERCEPTRON MODEL

The perceptron is a single-layer artificial neural network designed to perform binary classification. Its structure is simple yet mathematically powerful, consisting of input nodes, weights, a bias term, a summation unit, an activation function, and an output node. Understanding the structure of the perceptron is essential for implementing the model manually or using spreadsheet tools.

Components of the Perceptron

1. Input Features ($x_1, x_2, \dots, x_n$)

These represent measurable properties or attributes of the dataset. Each input corresponds to one feature of the problem. For example, in a student performance prediction model, inputs could include study hours and attendance percentage.

2. Weights ($w_1, w_2, \dots, w_n$)

Each input is multiplied by a corresponding weight. The weight determines

the importance or influence of that feature on the final output. ○ Large

positive weight → Strong positive influence ○ Large negative weight →

Strong negative influence ○ Zero weight → No influence

3. Bias (b)

The bias acts as an additional parameter that shifts the decision boundary. It increases the flexibility of the model and allows classification even when all input values are zero. Without bias, the decision boundary would always pass through the origin.

4. Summation Unit

The summation unit computes the linear combination of inputs and weights. It performs the following calculation:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

This value z is called the net input or weighted sum.

5. Activation Function

After computing the weighted sum, the perceptron applies a step activation function to convert the continuous value into a binary output.

6. Output (y)

The final output is either 0 or 1, representing the predicted class.

4. MATHEMATICAL MODEL OF THE PERCEPTRON

The mathematical model of the perceptron explains how input features are converted into a binary output using linear algebra and a threshold rule. The perceptron is a **linear classifier**, meaning it separates data using a straight line (or hyperplane) in the feature space.

Understanding this model is essential for manual implementation and spreadsheet-based learning.

4.1 Linear Combination of Inputs

The perceptron calculates a weighted sum of inputs along with a bias term:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Where:

- $x_1, x_2, \dots, x_n$ = Input features
- $w_1, w_2, \dots, w_n$ = Weights
- b = Bias
- z = Net input

Each weight determines the importance of its corresponding input. Positive weights increase influence, while negative weights reduce it.

4.2 Vector Form

The equation can be written in compact vector notation:

$$z = w^T x + b$$

Here, w is the weight vector and x is the input vector.

The dot product is:

$$w^T x = \sum_{i=1}^n w_i x_i$$

This representation simplifies computation, especially for higher-dimensional data.

4.3 Activation Function

After computing z , the perceptron applies the step function:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Thus, the complete model becomes:

$$y = f(w^T x + b)$$

The classification depends only on whether the weighted sum is positive or negative.

4.4 Decision Boundary

The decision boundary is defined by:

$$w^T x + b = 0$$

This equation represents:

- A line in 2D
- A plane in 3D
- A hyperplane in higher dimensions

Points on one side are classified as Class 1, and points on the other side as Class 0.

The weight vector is perpendicular to the boundary, and the bias shifts it.

5. ACTIVATION FUNCTION IN PERCEPTRON

The activation function is a crucial component of the perceptron model. After computing the weighted sum of inputs, the activation function determines the final output of the neuron.

Without an activation function, the perceptron would behave like a simple linear regression model. The activation function introduces a decision-making mechanism that enables classification.

5.1 Purpose of the Activation Function

The weighted sum

$$z = w^T x + b$$

produces a continuous numerical value. However, classification problems require discrete outputs such as 0 or 1. The activation function converts this continuous value into a binary result.

Thus, it acts as a threshold that decides whether the input belongs to Class 0 or Class 1.

5.2 Step Activation Function

The perceptron uses a **binary step function**, defined as:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

This means:

- If the weighted sum is positive or zero $\rightarrow$ Output = 1
- If the weighted sum is negative $\rightarrow$ Output = 0

The step function makes the perceptron a classifier instead of a regression model.

5.3 Threshold Representation

The activation function can also be written using a threshold value θ :

$$y = \begin{cases} 1 & \text{if } w^T x \geq \theta \\ 0 & \text{if } w^T x < \theta \end{cases}$$

Since bias $b = -\theta$, the threshold can be incorporated into the bias term. This shows that bias shifts the classification boundary.

5.4 Graphical Nature of Step Function

The step function is discontinuous at $z = 0$.

It abruptly changes from 0 to 1 at the threshold.

Because of this discontinuity:

- It is not differentiable at zero
- It has zero gradient elsewhere

This is why modern neural networks use differentiable activation functions such as Sigmoid or ReLU. However, for the basic perceptron learning algorithm, the step function is sufficient.

6. PERCEPTRON LEARNING LAW

The Perceptron Learning Law defines how the model updates its weights when it makes an incorrect prediction. This learning mechanism allows the perceptron to gradually adjust its decision boundary so that it correctly separates the two classes. The process is iterative and continues until the model achieves correct classification for all training samples (if the data is linearly separable).

6.1 Need for Weight Update

The perceptron output is given by:

$$y = f(w^T x + b)$$

If the predicted output y matches the target value t , no change is required.

If $y \neq t$, the weights must be adjusted.

6.2 Error Calculation

The classification error is defined as:

$$e = t - y$$

Possible cases:

- If $e = 0 \rightarrow$ Correct classification
- If $e = 1 \rightarrow$ Underprediction (model predicted 0 instead of 1)
- If $e = -1 \rightarrow$ Overprediction (model predicted 1 instead of 0)

Only when the error is non-zero do we update the weights.

6.3 Weight Update Rule

The perceptron updates weights using the formula:

$$w_i(new) = w_i(old) + \eta e x_i$$

Bias update:

$$b(new) = b(old) + \eta e$$

Where:

- η = Learning rate ($0 < \eta \leq 1$)
- e = Error
- x_i = Input feature

The learning rate controls how much the weights change in each update.

6.4 Interpretation of the Update Rule

Case 1: Correct Prediction ($e = 0$) No weight change occurs.

Case 2: Underprediction ($t = 1, y = 0$)

Weights increase in the direction of the input vector.

This moves the decision boundary toward correctly classifying the sample.

Case 3: Overprediction ($t = 0, y = 1$) Weights decrease.

This shifts the boundary in the opposite direction.

Thus, each update slightly rotates or shifts the decision boundary.

6.5 Training Algorithm Steps

1. Initialize weights and bias (usually zero).
2. For each training sample: ○ Compute weighted sum z

- Apply activation function ○
 - Calculate error e ○
 - Update weights and bias
3. Repeat for multiple epochs until:
- No misclassification occurs,
 - or ○ Maximum iterations
- are reached.

7. COMPLETE AND GATE TRAINING

To clearly understand the Perceptron Learning Law, we apply it step-by-step to train a perceptron for the AND logic gate. The AND gate is a binary classification problem that is linearly separable, making it suitable for perceptron training.

7.1 AND Gate Truth Table

x_1 x_2 Target (t)

0 0 0

0 1 0

1 0 0

1 1 1

The goal is to train the perceptron so that it correctly predicts the target output for all input combinations.

7.2 Initialization

Let initial weights and bias be:

$$w_1 = 0, w_2 = 0, b = 0$$

Learning rate:

$$\eta = 1$$

7.3 Epoch 1 Training

Sample 1: (0,0), Target = 0

$$z = 0$$

Output = 1 (since $z \geq 0$)

Error:

$$e = 0 - 1 = -1$$

Update:

$$w_1 = 0 + (1)(-1)(0) = 0 \quad w_2$$

$$= 0 + (1)(-1)(0) = 0$$

$$b = 0 + (1)(-1) = -1$$

Sample 2: (0,1), Target = 0

$$z = (0)(0) + (0)(1) - 1 = -1$$

Output = 0

Error = 0 $\rightarrow$ No update

Sample 3: (1,0), Target = 0

$$z = (0)(1) + (0)(0) - 1 = -1$$

Output = 0

No update

Sample 4: (1,1), Target = 1

$$z = (0)(1) + (0)(1) - 1 = -1$$

Output = 0 Error:

$$e = 1 - 0 = 1$$

Update:

$$w_1 = 0 + (1)(1)(1) = 1$$

$$w_2 = 0 + (1)(1)(1) = 1$$

$$b = -1 + 1 = 0$$

7.4 Further Training

By continuing this process for additional epochs, the weights gradually adjust. After convergence, one possible solution is:

$$w_1 = 1, w_2 = 1, b = -1.5$$

7.5 Final Decision Rule

$$z = x_1 + x_2 - 1.5$$

Classification rule:

- If $z \geq 0 \rightarrow \text{Output} = 1$
- Otherwise $\rightarrow \text{Output} = 0$

Only the input (1,1) produces output 1, correctly implementing the AND gate.

7.6 Observations

1. Weights change only when misclassification occurs.
2. The decision boundary gradually shifts.
3. Convergence occurs because AND gate data is linearly separable.
4. Manual calculation clearly demonstrates the learning process.

This example shows how the perceptron successfully learns a binary classification problem through iterative weight updates.

8. SPREADSHEET IMPLEMENTATION

In addition to manual calculation, the perceptron can be implemented using spreadsheet software such as Microsoft Excel or Google Sheets. Spreadsheet implementation is highly useful for academic understanding because each calculation step is visible and easy to verify.

8.1 Spreadsheet Layout

Create the following columns:

| x_1 | x_2 | w_1 | w_2 | b | z | y | Target (t) | Error (e) |

Each row represents one training sample.

Weights and bias can be updated row-by-row to clearly observe learning progression.

8.2 Enter Initial Values Initialize:

- $w_1 = 0$
- $w_2 = 0$
- $b = 0$
- Learning rate $\eta = 1$

Enter AND gate dataset:

x_1 x_2 Target

0 0 0

0 1 0

1 0 0

1 1 1

8.3 Excel Formulas

1. Weighted Sum (z)

$$z = w_1x_1 + w_2x_2 + b$$

Excel formula:

$$= (C2*A2) + (D2*B2) + E2$$

2. Activation Function (y)

$$= \text{IF}(F2 \geq 0, 1, 0)$$

This applies the step function.

3. Error Calculation

$$e = t - y$$

Excel formula:

$$= H2 - G2$$

4. Weight Update

$$w_1(\text{new}) = w_1(\text{old}) + \eta ex_1$$

Similarly for w_2 :

$$w_2(\text{new}) = w_2(\text{old}) + \eta ex_2$$

Bias update:

$$b(\text{new}) = b(\text{old}) + \eta e$$

These formulas can be implemented in adjacent cells.

8.4 Training Process in Spreadsheet Two

methods can be used:

Method 1: Row-by-Row Update

Weights are updated immediately after each training sample.

Method 2: Epoch-Based Update

Weights remain fixed for one full pass through the dataset, then updated.

For learning purposes, row-by-row update clearly shows weight adjustments.

9. DECISION BOUNDARY

The decision boundary is the line (or hyperplane) that separates two classes in the feature space.

It is defined by:

$$w_1x_1 + w_2x_2 + b = 0$$

9.1 Geometrical Meaning

- In 2D → Straight line
- In 3D → Plane
- In n-dimensions → Hyperplane

Points on one side of the boundary are classified as Class 1, and points on the other side as Class 0.

9.2 Role of Weights and Bias

- Weights determine slope and orientation.
- Bias shifts the boundary parallelly.

Changing weights rotates the boundary.

Changing bias moves it without changing its direction.

10. ADVANTAGES AND LIMITATIONS OF THE PERCEPTRON

The perceptron is an important foundational model in machine learning. While it has several strengths, it also has notable limitations that led to the development of more advanced neural network models.

10.1 Advantages

1. Simple and Easy to Understand

The perceptron has a straightforward mathematical structure. Its working mechanism is easy to explain and implement manually or using spreadsheets.

2. Computationally Efficient

The algorithm requires only basic arithmetic operations such as multiplication, addition, and comparison. This makes it fast and suitable for small-scale problems.

3. Guaranteed Convergence (for Linearly Separable Data)

According to the Perceptron Convergence Theorem, if the dataset is linearly separable, the algorithm will converge in a finite number of iterations.

4. Foundation for Neural Networks

The perceptron introduced key concepts such as weighted inputs, bias, activation function, and error-driven learning. These principles are used in modern neural networks and deep learning architectures.

5. Educational Value

It is highly useful for understanding core ideas of supervised learning, decision boundaries, and iterative weight updates.

10.2 Limitations

1. Only Linearly Separable Problems

The perceptron cannot solve problems where classes cannot be separated by a straight line. The classic example is the XOR problem.

2. Binary Classification Only

A basic perceptron can classify only two classes at a time. Multi-class classification requires extensions.

3. **Non-Differentiable Activation Function**

The step function is not differentiable, which makes it unsuitable for gradient-based optimization methods used in deep learning.

4. **Limited Representational Power**

Being a single-layer model, it cannot learn complex patterns or hierarchical feature representations.

5. **Sensitive to Learning Rate**

A very large learning rate may cause unstable updates, while a very small learning rate may slow down convergence.

11. CONCLUSION

The perceptron is one of the earliest and most significant algorithms in the field of artificial neural networks. Developed by Frank Rosenblatt, it introduced the revolutionary idea that machines can learn from data by adjusting weights based on classification errors. Despite its simplicity, the perceptron established the fundamental mathematical framework for modern neural network models.

In this report, the perceptron model was studied from both theoretical and practical perspectives. The mathematical formulation, including the weighted sum equation and step activation function, was explained in detail. The Perceptron Learning Law was applied manually to train the model for the AND gate problem. Through iterative updates of weights and bias, the model successfully learned to classify linearly separable data.

Although the perceptron is limited to linearly separable problems and cannot handle complex non-linear datasets, it remains a foundational learning model. Its concepts form the basis for multi-layer neural networks and modern deep learning systems.

In conclusion, mastering the perceptron strengthens understanding of supervised learning, decision boundaries, and iterative optimization. It serves as an essential stepping stone toward advanced neural network architectures and artificial intelligence applications.

12. REFERENCES

1. Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson